Capstone Lab Task 1: ADC

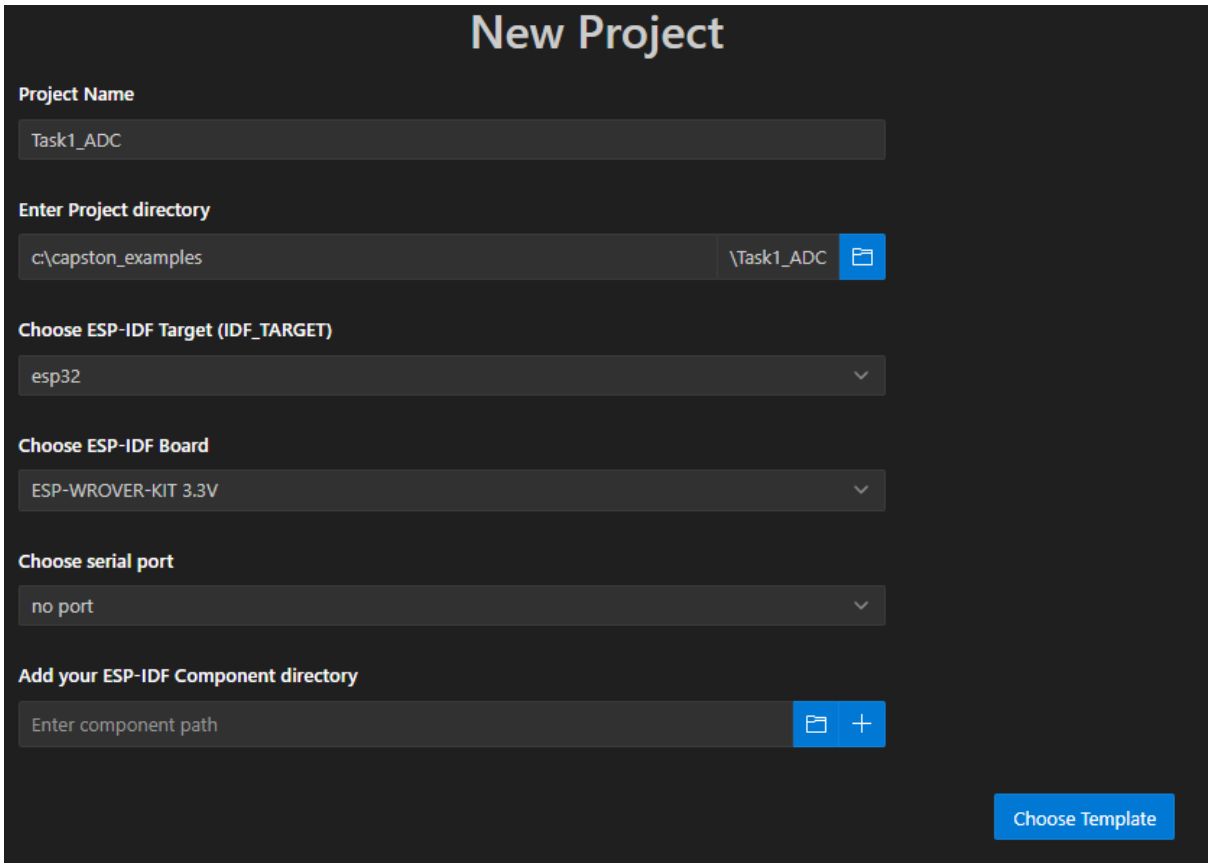
The objective of the first task in the Capstone lab is to measure the voltage on a pin of the ESP32 board. This measurement is performed using an ADC, and the values are sent over UART to the terminal. First, we intend to explain ADC concepts in ESP32.

Before implementing this task, it is essential to become familiar with the basic operation of the ESP32 ADC. This includes understanding key parameters such as resolution (bit width), input attenuation, and the available operating modes, including one-shot and continuous modes. To support this, the following reference can be used to learn more about these ADC characteristics.

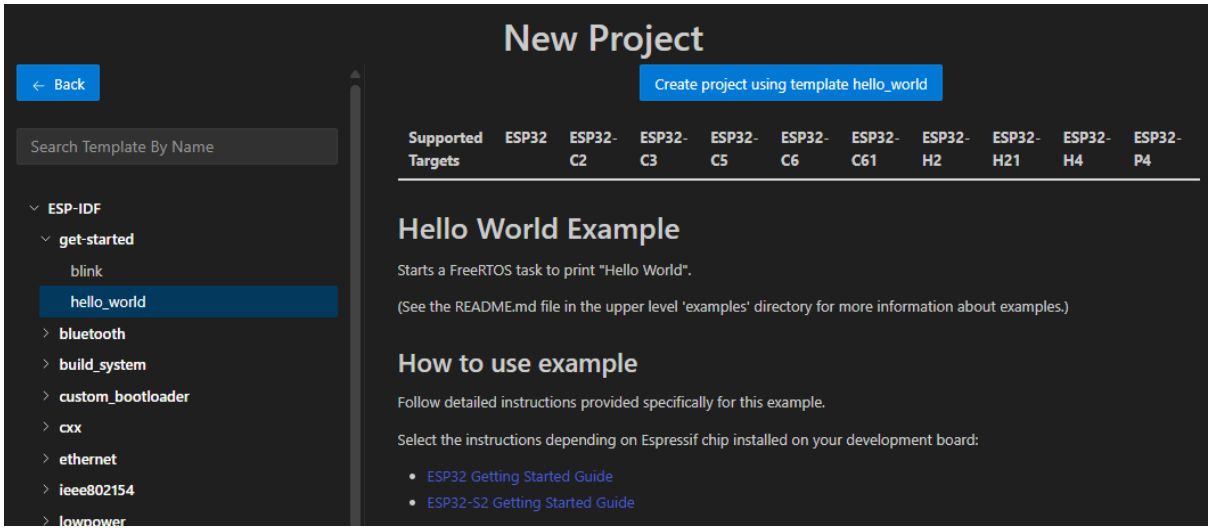
https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc/index.html

There are several methods to write a program for the ESP32. Here, we use the C language, the VSCode IDE, and the ESP-IDF library to develop our code on the module. To begin, a new project should be created for the implementation.

View → Command Palette → ESP-IDE: Create New Project → Select ESP-IDE version.



The Hello World template can be used as a starting point for the project. Alternatively, the oneshot_read example provided in the ESP-IDF under peripherals → adc can be used. In this case, the required ADC libraries and header files are included automatically.



If you create a project from the Hello World example, you will need to add the ADC library to the project. Therefore, open the CMakelists.txt file under the main folder (not the one for the whole project) and add esp_adc after PRIV_REQUIRES.

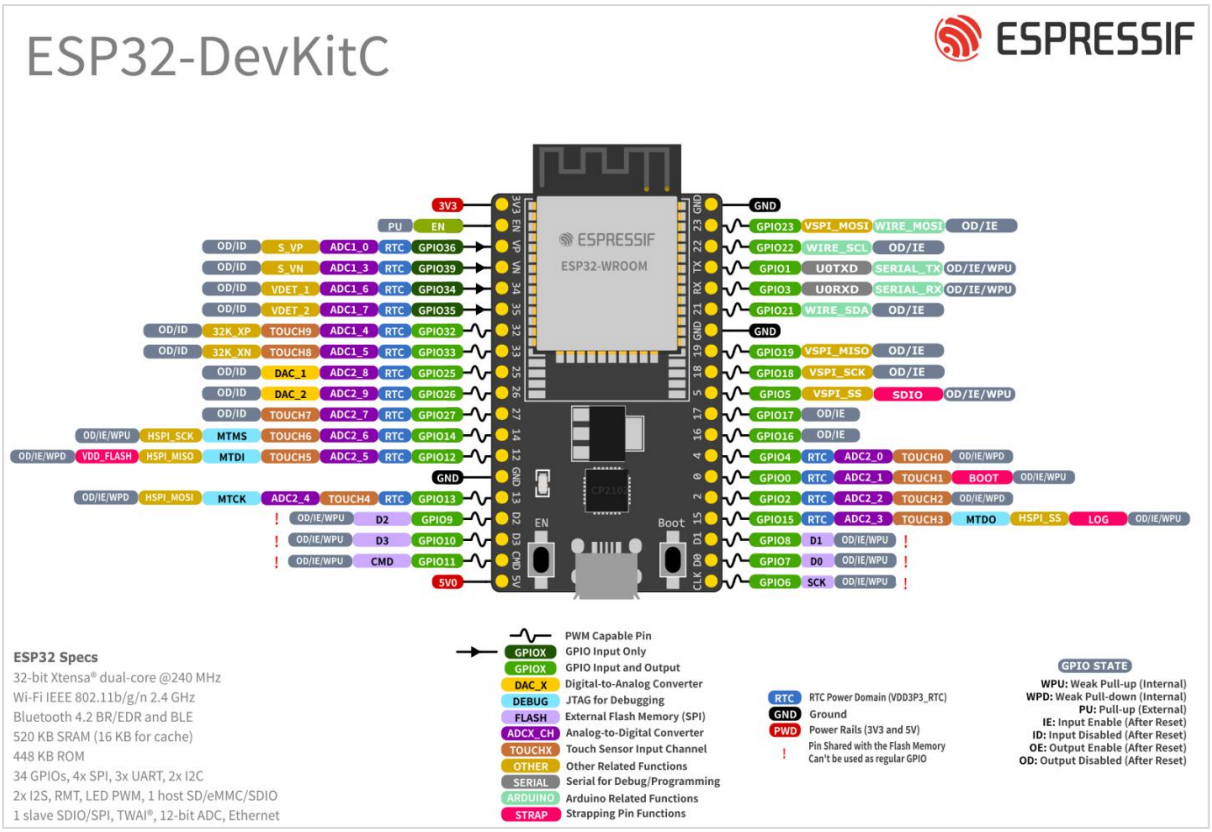
```
main > M CMakeLists.txt
idf_component_register(SRCS "hello_world_main.c"
                       PRIV_REQUIRES spi_flash esp_adc
                       INCLUDE_DIRS "")
```

You will need adc_oneshot.h for using the ADC in oneshot mode, and adc_cali.h and adc_cali_scheme.h for calibration. We should add these header files to the main file.

```
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
```

The ADC is configured to measure the voltage on a specific pin, which is selected based on the ADC unit and channel mapping. For example, GPIO34 corresponds to ADC Unit 1, Channel 6, and is configured accordingly. The ADC channels, units, and their corresponding Pins are illustrated in the following figure. In addition, other ADC parameters must be configured, including the resolution, sampling frequency, and input voltage range.

Finally, please note that the device uses 2 MB of flash memory by default. However, our ESP32 board has 4 MB of flash memory. You can change the device hardware configuration according to the “Getting Started” document.



Task 1.a:

Write a program to read the voltage on GPIO34 in one-shot mode with the following specification:

- Sampling rate: 100 KHz
- Maximum Voltage range
- Resolution: 12-bit

Test the device and verify it works correctly.

Task 1.b:

Averaging is a commonly used technique for smoothing measured values and reducing the effects of noise. Implement averaging on N values in the written code. Change the N value and observe the impact of noise.

Task1.c (Optional):

Extend the code to enable automatic ADC calibration and use the built-in ESP-IDF function to convert raw ADC readings to voltage values. Then, observe and discuss the changes in the measured results, with particular attention to the minimum and maximum voltage values obtained.